

LTE Standard(U)系列 QuecOpen(SDK)

SPI NAND Flash API 参考手册

LTE Standard 模块系列

版本：1.1

日期：2023-09-07

状态：受控文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司

上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233

电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：

<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬软件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2023，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2023.

文档历史

修订记录

版本	日期	作者	变更表述
-	2021-03-02	Ryan YI	文档创建
1.0	2021-08-27	Ryan YI	受控版本
1.1	2023-09-07	Sum LI	1. 基于 QuecOpen 方案统一命名，更新文档名称。 2. 新增适用模块 EC200G-CN、EC600G 系列、EC800G-CN、EG700G-CN、EG800G 系列、EG912U-GL 和 EG915U 系列。 3. 新增 SPI 模式及其引脚定义相关备注（第 1.1 章）。 4. 新增模块 SPI NAND flash 型号适配相关信息（第 2 章）。 5. 更新 SPI NAND flash 结果码枚举 <code>ql_errcode_spi_nand_e</code> 定义（第 3.1.3.1.1 章）。 6. 更新 SPI 时钟频率枚举 <code>ql_spi_clk_e</code> 定义（第 3.1.3.1.3 章）。 7. 新增通用 SPI NAND flash API 函数（第 3.1.3.4、3.1.3.7 章）： <code>ql_spi_nand_read_devid_ex()</code>; <code>ql_spi_nand_write_spare()</code>。 8. 新增专用 SPI NAND flash API 函数(第 3.1.3.12~3.1.3.22 章)： <code>ql_spi6_nand_init()</code>; <code>ql_spi6_nand_init_ext()</code>; <code>ql_spi6_nand_read_devid()</code>; <code>ql_spi6_nand_read_devid_ex()</code>; <code>ql_spi6_nand_read_page_spare()</code>; <code>ql_spi6_nand_write_page_spare()</code>; <code>ql_spi6_nand_write_spare()</code>; <code>ql_spi6_nand_read_status()</code>; <code>ql_spi6_nand_write_status()</code>; <code>ql_spi6_nand_erase_block()</code>; <code>ql_spi6_nand_reset()</code>。 9. 新增 SPI NAND flash 文件系统挂载 API 函数（第 3.2 章）： <code>ql_nand_flash_mkfs()</code>; <code>ql_nand_flash_umount()</code>;

`ql_nand_flash_mount();`

`ql_nand_get_hw_info()。`

10. 新增将 NOR flash 更换成 NAND flash 进行测试的注意事项(第 4 章)。

11. 新增 SPI NAND flash 文件系统挂载 API 开发示例和功能调试(第 4.1.2、4.2.2 章)。

目录

文档历史.....	3
目录.....	5
表格索引.....	7
图片索引.....	8
1 引言.....	9
1.1. 适用模块.....	9
2 SPI NAND Flash 型号适配.....	11
2.1. 支持的 NAND Flash 型号信息.....	11
2.2. NAND Flash 相关参数含义.....	12
2.2.1. quec_spi_nand_flash_info_s.....	12
2.2.1.1. quec_spi_nand_spare_info_s.....	13
2.2.1.2. quec_spi_nand_flash_id_type_e.....	14
2.2.1.3. NAND ID.....	14
2.3. 获取和配置参数.....	14
3 SPI NAND Flash API.....	17
3.1. 通用和专用 SPI NAND Flash API.....	17
3.1.1. 头文件.....	17
3.1.2. 函数概览.....	17
3.1.3. 函数详解.....	18
3.1.3.1. ql_spi_nand_init.....	18
3.1.3.1.1. ql_errcode_spi_nand_e.....	19
3.1.3.1.2. ql_spi_port_e.....	20
3.1.3.1.3. ql_spi_clk_e.....	21
3.1.3.1.4. ql_spi_input_sel_e.....	23
3.1.3.1.5. ql_spi_transfer_mode_e.....	24
3.1.3.1.6. ql_spi_cs_sel_e.....	24
3.1.3.2. ql_spi_nand_init_ext.....	25
3.1.3.2.1. ql_spi_nand_config_s.....	25
3.1.3.3. ql_spi_nand_read_devid.....	26
3.1.3.4. ql_spi_nand_read_devid_ex.....	26
3.1.3.4.1. ql_spi_nand_id_type_e.....	27
3.1.3.5. ql_spi_nand_read_page_spare.....	27
3.1.3.6. ql_spi_nand_write_page_spare.....	28
3.1.3.7. ql_spi_nand_write_spare.....	29
3.1.3.8. ql_spi_nand_read_status.....	29
3.1.3.8.1. ql_spi_nand_status_reg_e.....	30
3.1.3.9. ql_spi_nand_write_status.....	31
3.1.3.10. ql_spi_nand_erase_block.....	31
3.1.3.11. ql_spi_nand_reset.....	32
3.1.3.12. ql_spi6_nand_init.....	32

3.1.3.12.1.	ql_spi6_nand_flash_clk_e	33
3.1.3.13.	ql_spi6_nand_init_ext	35
3.1.3.13.1.	ql_spi6_nand_config_s.....	35
3.1.3.14.	ql_spi6_nand_read_devid	36
3.1.3.15.	ql_spi6_nand_read_devid_ex	36
3.1.3.16.	ql_spi6_nand_read_page_spare.....	37
3.1.3.17.	ql_spi6_nand_write_page_spare	38
3.1.3.18.	ql_spi6_nand_write_spare	38
3.1.3.19.	ql_spi6_nand_read_status	39
3.1.3.20.	ql_spi6_nand_write_status.....	39
3.1.3.21.	ql_spi6_nand_erase_block	40
3.1.3.22.	ql_spi6_nand_reset.....	40
3.2.	SPI NAND Flash 文件系统挂载 API.....	41
3.2.1.	头文件	41
3.2.2.	函数概览	41
3.2.3.	函数详解	41
3.2.3.1.	ql_nand_flash_mkfs	41
3.2.3.1.1.	ql_errcode_nand_flash_e	42
3.2.3.2.	ql_nand_flash_umount.....	43
3.2.3.3.	ql_nand_flash_mount.....	43
3.2.3.4.	ql_nand_get_hw_info	44
3.2.3.4.1.	ql_nand_hw_info_t	44
4	示例	45
4.1.	开发示例	45
4.1.1.	通用 SPI NAND Flash API 开发示例	45
4.1.2.	SPI NAND Flash 文件系统挂载 API 开发示例	46
4.2.	功能调试	46
4.2.1.	通用 SPI NAND Flash API 功能调试	46
4.2.2.	SPI NAND Flash 文件系统挂载 API 功能调试	47
5	附录 参考文档及术语缩写	49

表格索引

表 1: 适用模块	9
表 2: 函数概览	17
表 3: 函数概览	41
表 4: 参考文档	49
表 5: 术语缩写	49

图片索引

图 1: 支持的 NAND Flash 型号相关参数	11
图 2: GD5F1GQ5 阵列组织结构（硬件 ECC 开启）	15
图 3: GD5F1GQ5 ECC 保护和备用区	16
图 4: 入口函数 <i>ql_spi_nand_flash_demo_init()</i>	45
图 5: 通用 SPI NAND Flash API 示例默认不启动	45
图 6: 入口函数 <i>ql_fs_nand_flash_demo_init()</i>	46
图 7: SPI NAND Flash 文件系统挂载 API 示例默认不启动	46
图 8: COM 设备（ECx00U 系列/EGx00U 系列/EG91xU 系列模块）	46
图 9: COM 设备（ECx00G 系列/EGx00G 系列模块）	47
图 10: Log 信息（通用 SPI NAND Flash）	47
图 11: Log 信息（SPI NAND Flash 文件系统挂载）	48

1 引言

移远通信 LTE Standard(U)系列模块支持 QuecOpen®方案；QuecOpen®是基于 RTOS 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考[文档 \[1\]](#)。

本文档适用于 CSDK 构建环境的 QuecOpen®方案，主要介绍 LTE Standard(U)系列模块的 SPI NAND flash 型号适配、通用及专用 SPI NAND flash API、SPI NAND flash 文件系统挂载 API，及相关开发示例和功能调试。

1.1. 适用模块

表 1：适用模块

模块系列	模块
ECx00G	EC200G-CN
	EC600G 系列
	EC800G-CN
ECx00U	EC200U 系列
	EC600U 系列
EGx00U	EG500U-CN
	EG700U-CN
EGx00G	EG700G-CN
	EG800G 系列
EG91xU	EG912U-GL
	EG915U 系列

备注

1. 本文档不适用于 EC200D 系列模块。
2. 本文档内，SPI 包含 Standard SPI、Dual SPI 和 Quad SPI 三种模式，其中 Standard SPI 为通用 SPI（4 线）、Dual SPI 和 Quad SPI 为专用 SPI（6 线）。通用和专用 SPI 的控制器不同，因此分别使用不同的驱动代码。不同模式 SPI 定义引脚如下：
 - Standard SPI: SCLK、CS#、SI 和 SO;
 - Dual SPI: SCLK、CS#、SIO0 和 SIO1;
 - Quad SPI: SCLK、CS#、SIO0、SIO1、SIO2 和 SIO3。

2 SPI NAND Flash 型号适配

模块 QuecOpen CSDK 代码中提供了 NAND flash 配置文件 *quec_spi_nand_flash_prop.c* 和 *quec_spi_nand_flash_prop.h*，位于 `\components\ql_kernel\drivers\` 目录下。用户可在配置文件中自行添加相应的 NAND flash 型号信息。

本章节主要介绍 NAND flash 型号的相关参数信息及如何添加新型号 NAND flash。

2.1. 支持的 NAND Flash 型号信息

模块支持的部分 NAND flash 型号信息已在 *quec_spi_nand_flash_prop.c* 中的 *quec_spi_nand_flash_props* 内定义，例如：

```

/* 4-line SPI NAND Flash supported model list, can add by yourself */
static quec_spi_nand_flash_info_s quec_spi_nand_flash_props[] =
{
    // You can cut out it by defining macros
    // #define DISABLE_W25N02JWXXIF
    #ifndef DISABLE_W25N02JWXXIF
    {
        // W25N02JWxxIF/IC, MID=EFBF22, 2--2G
        .page_totalsize = 2112,
        .page_mainsize = 2048,
        .page_sparesize = 64,
        .block_pagenum = 64,
        .block_totalnum = 2048,
        .cache_blocknum = 5,
        {
            .page_spare_shift = 0, // page_mainsize = 2048
            .block_postion_shift = 2, // 2050
            .block_type_shift = 3, // 2051
            .logic_addr_shift = 16, // 2064
            .page_used_shift = 32, // 2080
            .page_garbage_shift = 33, // 2081
            .page_num_shift = 34, // 2082
        },
        .nand_id = 0xEF0F, // bit[7:4] don't compare, should be 0
        .nand_id_type = QUEC_NAND_FLASH_ID_24BIT,
    },
    #endif
    #ifndef DISABLE_GD5F1GQ5XEXXG
    {
        // GD5F1GQ5XExxG, MID=C841/C851, 1--1G
        .page_totalsize = 2112,
        .page_mainsize = 2048,
        .page_sparesize = 64,
        .block_pagenum = 64,
        .block_totalnum = 1024,
        .cache_blocknum = 5,
        {
            .page_spare_shift = 0, // page_mainsize = 2048
            .block_postion_shift = 2, // 2050
            .block_type_shift = 3, // 2051
            .logic_addr_shift = 16, // 2064
            .page_used_shift = 32, // 2080
            .page_garbage_shift = 33, // 2081
            .page_num_shift = 34, // 2082
        },
        .nand_id = 0xC801, // bit[7:4] don't compare, should be 0
        .nand_id_type = QUEC_NAND_FLASH_ID_16BIT,
    },
    #endif
};

```

图 1：支持的 NAND Flash 型号相关参数

2.2. NAND Flash 相关参数含义

2.2.1. quec_spi_nand_flash_info_s

`quec_spi_nand_flash_props`（其类型 `quec_spi_nand_flash_info_s` 在 `quec_spi_nand_flash_prop.h` 中定义）包含了 NAND flash 型号的相关参数，具体如下：

```
typedef struct
{
    unsigned int page_totalsize;
    unsigned int page_mainsize;
    unsigned int page_sparesize;
    unsigned int block_pagenum;
    unsigned int block_totalnum;
    unsigned int cache_blocknum;
    quec_spi_nand_spare_info_s spare_info;
    unsigned int nand_id;
    quec_spi_nand_flash_id_type_e nand_id_type;
} quec_spi_nand_flash_info_s
```

● 参数

类型	参数	描述
unsigned int	<code>page_totalsize</code>	每页的总大小（即主区域和备用区总大小）。单位：字节。
unsigned int	<code>page_mainsize</code>	每页主区域的大小。单位：字节。
unsigned int	<code>page_sparesize</code>	每页备用区的大小。单位：字节。
unsigned int	<code>block_pagenum</code>	块中的页数量。
unsigned int	<code>block_totalnum</code>	flash 中的块数量。
unsigned int	<code>cache_blocknum</code>	缓存块的数量，此值不需要修改。
<code>quec_spi_nand_spare_info_s</code>	<code>spare_info</code>	备用区配置参数；详见第2.2.1.1章。
unsigned int	<code>nand_id</code>	NAND ID；详见第2.2.1.3章。
<code>quec_spi_nand_flash_id_type_e</code>	<code>nand_id_type</code>	JEDEC ID 类型；详见第2.2.1.2章。

2.2.1.1. quec_spi_nand_spare_info_s

NAND flash 备用区配置参数结构体定义如下：

```
typedef struct
{
    unsigned short page_spare_shift;
    unsigned short block_postion_shift;
    unsigned short block_type_shift;
    unsigned short logic_addr_shift;
    unsigned short page_used_shift;
    unsigned short page_garbage_shift;
    unsigned short page_num_shift;
} quec_spi_nand_flash_info_s
```

● 成员

类型	成员	描述
unsigned short	<i>page_spare_shift</i>	备用区中坏块标记的偏移地址，占用 2 字节。
unsigned short	<i>block_postion_shift</i>	备用区中当前缓存块位置标记的偏移地址，占用 1 字节。
unsigned short	<i>block_type_shift</i>	备用区中块类型标记的偏移地址，占用 1 字节。
unsigned short	<i>logic_addr_shift</i>	备用区中逻辑块地址标记的偏移地址，占用 4 字节。
unsigned short	<i>page_used_shift</i>	备用区中已使用页标记的偏移地址，占用 1 字节。
unsigned short	<i>page_garbage_shift</i>	备用区中页回收标记的偏移地址，占用 1 字节。
unsigned short	<i>page_num_shift</i>	备用区中页号标记的偏移地址，占用 2 字节。

备注

1. 硬件 ECC 校验功能默认开启（不支持软件 ECC 校验）。配置 *quec_spi_nand_flash_info_s* 结构体中的字段时，必须选择备用区中无 ECC 保护的区域，否则 ECC 校验将报错。需要注意，不同型号的 NAND flash 备用区中**无 ECC 保护的区域**占用字节和所处位置差异较大，必须保证该区域占用 12 字节以上。
2. *page_spare_shift* 是标记坏块的字段，设置备用区中坏块标记的偏移地址时，需要和 NAND flash 芯片中规定的标记坏块的字段保持一致。其他字段可视需求调整，但**必须保证每个多字节字段所占用的地址是连续的**。

2.2.1.2. quec_spi_nand_flash_id_type_e

JEDEC ID 类型枚举定义如下：

```
typedef enum
{
    QUEC_NAND_FLASH_ID_16BIT,
    QUEC_NAND_FLASH_ID_24BIT,
}quec_spi_nand_flash_id_type_e
```

● 成员

成员	描述
<i>QUEC_NAND_FLASH_ID_16BIT</i>	16 位 JEDEC ID，即使用 9Fh 命令读取到的 JEDEC ID，由厂商 ID（8 位）和设备 ID（8 位）组成。
<i>QUEC_NAND_FLASH_ID_24BIT</i>	24 位 JEDEC ID，即使用 9Fh 命令读取到的 JEDEC ID，由厂商 ID（8 位）和设备 ID（16 位）组成。

2.2.1.3. NAND ID

NAND ID 是额外定义的一个仅用于区分不同型号 NAND flash 的 ID，有别于 JEDEC ID。

对于同一主型号 NAND flash 的不同子型号，其 JEDEC ID 可能稍有差异。实际应用时，用户只需区分主型号，因此不直接使用 JEDEC ID 区分不同型号，而使用本章介绍的 NAND ID 进行区分。可按照以下方法将 JEDEC ID 转换为 NAND ID：

取 16 位或 24 位 JEDEC ID 的高 16 位，并将高 16 位中 bit [7:4]赋值为 0。

例如，通过 *ql_spi_nand_read_devid_ex()*（详见第 3.1.3.4 章）读取的 W25N02JW JEDEC ID（24 位）为 EFBF22，取其高 16 位，且 bit [7:4]全赋值为 0，将得到 NAND ID 为 0xEF0F；读取的 GD5F1GQ5 JEDEC ID 为 0xC841 或 0xC851，转换后的 NAND ID 为 0xC801。

2.3. 获取和配置参数

本章以 GD5F1GQ5 为例，介绍如何获取并配置新的 NAND flash 型号信息。

1. 参考 GD5F1GQ5 的 NAND flash 芯片数据手册，获取所需配置的参数：

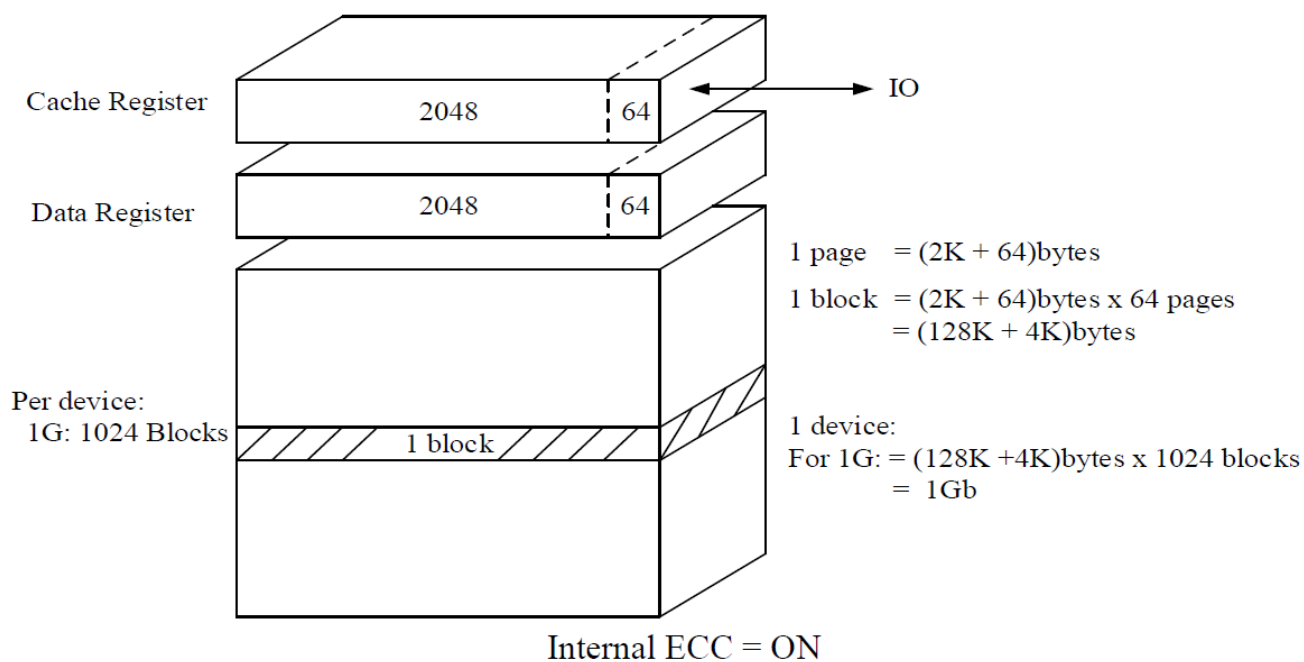


图 2: GD5F1GQ5 阵列组织结构（硬件 ECC 开启）

由上图可知：

$page_totalsize = 2\text{ KB} + 64\text{ B} = 2112\text{ B}$

$page_mainsize = 2\text{ KB} = 2048\text{ B}$

$page_sparesize = 64\text{ B}$

$block_pagenum = 64$

$block_totalnum = 1024$

$cache_blocknum = 5$ （此参数统一为 5，无需修改。）

Main Area(2KB)				Spare Area(128B)							
User data				User meta data(I+II)				ECC Parity Data			
Main0	Main1	Main2	Main3	Spare0	Spare1	Spare2	Spare3	Spare0	Spare1	Spare2	Spare3
(512B)	(512B)	(512B)	(512B)	(4B+12B)	(4B+12B)	(4B+12B)	(4B+12B)	(16B)	(16B)	(16B)	(16B)
Max Byte Address	Min Byte Address			ECC Protected	Area		Description				
1FFH	000H			Yes	Main 0		User data 0				
3FFH	200H			Yes	Main 1		User data 1				
5FFH	400H			Yes	Main 2		User data 2				
7FFH	600H			Yes	Main 3		User data 3				
803H	800H			No	Spare 0		User meta 0 data I ⁽¹⁾				
80FH	804H			Yes	Spare 0		User meta 0 data II				
813H	810H			No	Spare 1		User meta 1 data I				
81FH	814H			Yes	Spare 1		User meta 1 data II				
823H	820H			No	Spare 2		User meta 2 data I				
82FH	824H			Yes	Spare 2		User meta 2 data II				
833H	830H			No	Spare 3		User meta 3 data I				
83FH	834H			Yes	Spare 3		User meta 3 data II				
84FH	840H			Yes	Spare 0		ECC Parity Data				
85FH	850H			Yes	Spare 1		ECC Parity Data				
86FH	860H			Yes	Spare 2		ECC Parity Data				
87FH	870H			Yes	Spare 3		ECC Parity Data				

图 3: GD5F1GQ5 ECC 保护和备用区

根据上图中无 ECC 保护的区域分配，可获取如下参数：

```

page_spare_shift = 0
block_postion_shift = 2
block_type_shift = 3
logic_addr_shift = 16
page_used_shift = 32
page_garbage_shift = 33
page_num_shift = 34

```

有关如何获取 *nand_id* 和 *nand_id_type*，详见第 2.2.1.2 和 2.2.1.3 章。

- 参考图 1，在 *quec_spi_nand_flash_prop.c* 中的 *quec_spi_nand_flash_props* 内添加新型号 NAND flash 信息。

3 SPI NAND Flash API

3.1. 通用和专用 SPI NAND Flash API

3.1.1. 头文件

通用和专用 SPI NAND flash API 头文件为 `ql_api_spi_nand_flash.h`，位于 SDK 包的 `components\ql-kernel\incl` 目录下。若无特别说明，本文档所述头文件均在该目录下。

3.1.2. 函数概览

表 2：函数概览

函数	说明
<code>ql_spi_nand_init()</code>	初始化配置通用 SPI（仅配置 SPI 总线和时钟频率）
<code>ql_spi_nand_init_ext()</code>	初始化配置通用 SPI
<code>ql_spi_nand_read_devid()</code>	读取通用 SPI NAND flash 的 JEDEC ID
<code>ql_spi_nand_read_devid_ex()</code>	根据类型读取通用 SPI NAND flash 的 JEDEC ID
<code>ql_spi_nand_read_page_spare()</code>	读取通用 SPI NAND flash 的页和备用区数据
<code>ql_spi_nand_write_page_spare()</code>	写入数据到通用 SPI NAND flash 的页和备用区
<code>ql_spi_nand_write_spare()</code>	写入数据到通用 SPI NAND flash 的备用区
<code>ql_spi_nand_read_status()</code>	读取通用 SPI NAND flash 状态寄存器的值
<code>ql_spi_nand_write_status()</code>	写入通用 SPI NAND flash 状态寄存器的值
<code>ql_spi_nand_erase_block()</code>	擦除通用 SPI NAND flash 的块
<code>ql_spi_nand_reset()</code>	复位通用 SPI NAND flash
<code>ql_spi6_nand_init()</code>	初始化配置专用 SPI（仅配置 SPI 总线、时钟频率和时钟分频系数）

<code>ql_spi6_nand_init_ext()</code>	初始化配置专用 SPI
<code>ql_spi6_nand_read_devid()</code>	读取专用 SPI NAND flash 的 JEDEC ID
<code>ql_spi6_nand_read_devid_ex()</code>	根据类型读取专用 SPI NAND flash 的 JEDEC ID
<code>ql_spi6_nand_read_page_spare()</code>	读取专用 SPI NAND flash 的页和备用区数据
<code>ql_spi6_nand_write_page_spare()</code>	写入数据到专用 SPI NAND flash 的页和备用区
<code>ql_spi6_nand_write_spare()</code>	写入数据到专用 SPI NAND flash 的备用区
<code>ql_spi6_nand_read_status()</code>	读取专用 SPI NAND flash 状态寄存器的值
<code>ql_spi6_nand_write_status()</code>	写入专用 SPI NAND flash 状态寄存器的值
<code>ql_spi6_nand_erase_block()</code>	擦除专用 SPI NAND flash 的块
<code>ql_spi6_nand_reset()</code>	复位专用 SPI NAND flash

3.1.3. 函数详解

3.1.3.1. ql_spi_nand_init

该函数用于初始化配置通用 SPI(仅配置 SPI 总线和时钟频率)。调用该函数前,需通过 `ql_pin_set_func()` 设置相关 GPIO 引脚为通用 SPI 功能,详情请参考[文档 \[2\]](#)。

调用该函数将默认配置 `ql_spi_input_sel_e` 为 `QL_SPI_DI_1`, `ql_spi_transfer_mode_e` 为 `QL_SPI_DIRECT_POLLING`, `ql_spi_cs_sel_e` 为 `QL_SPI_CS0`。相关参数描述详情请参考[第 3.1.3.1.4 章](#)、[第 3.1.3.1.5 章](#)和[第 3.1.3.1.6 章](#)。

● 函数原型

```
ql_errcode_spi_nand_e ql_spi_nand_init(ql_spi_port_e port, ql_spi_clk_e spiclk)
```

● 参数

`port`:

[In] SPI 总线; 详见[第 3.1.3.1.2 章](#)。

`spiclk`:

[In] SPI 时钟频率; 详见[第 3.1.3.1.3 章](#)。

- 返回值

详见第3.1.3.1.1章。

备注

若选择了错误的 SPI 总线进行初始化，将导致 NAND flash 读、写、擦除操作失败。因此，需对初始化返回值进行判定，若判定为初始化失败，应避免进行后续读、写、擦除操作。

3.1.3.1.1. ql_errcode_spi_nand_e

SPI NAND flash 结果码枚举定义如下：

```
typedef ql_errcode_spi_flash_e ql_errcode_spi_nand_e
```

```
typedef enum
{
    QL_SPI_FLASH_SUCCESS                = 0,
    QL_SPI_FLASH_ERROR                  = 1 |
(QL_COMPONENT_STORAGE_EXTFLASH << 16),
    QL_SPI_FLASH_PARAM_TYPE_ERROR,
    QL_SPI_FLASH_PARAM_DATA_ERROR,
    QL_SPI_FLASH_PARAM_ACQUIRE_ERROR,
    QL_SPI_FLASH_PARAM_NULL_ERROR,
    QL_SPI_FLASH_DEV_NOT_ACQUIRE_ERROR,
    QL_SPI_FLASH_PARAM_LENGTH_ERROR,
    QL_SPI_FLASH_MALLOC_MEM_ERROR,
    QL_SPI_FLASH_NOT_FLASH_CONN_ERROR,
    QL_SPI_FLASH_NOT_SUPPORT_ERROR,
    QL_SPI_FLASH_OP_NOT_SUPPORT_ERROR,
    QL_SPI_FLASH_ECC_ERROR,
    QL_SPI_FLASH_PROGRAM_ERROR,
    QL_SPI_FLASH_ERASE_ERROR,
    QL_SPI_FLASH_MUTEX_CREATE_ERROR,
    QL_SPI_FLASH_MUTEX_LOCK_ERROR,
    QL_SPI_FLASH_SET_GPIO_ERROR
}ql_errcode_spi_flash_e
```

- 成员

成员	描述
QL_SPI_FLASH_SUCCESS	函数执行成功

QL_SPI_FLASH_ERROR	SPI 总线其他错误
QL_SPI_FLASH_PARAM_TYPE_ERROR	参数类型错误
QL_SPI_FLASH_PARAM_DATA_ERROR	参数数据错误
QL_SPI_FLASH_PARAM_ACQUIRE_ERROR	参数无法获取
QL_SPI_FLASH_PARAM_NULL_ERROR	参数 NULL 错误
QL_SPI_FLASH_DEV_NOT_ACQUIRE_ERROR	无法获取 SPI 总线
QL_SPI_FLASH_PARAM_LENGTH_ERROR	参数长度错误
QL_SPI_FLASH_MALLOC_MEM_ERROR	申请内存错误
QL_SPI_FLASH_NOT_FLASH_CONN_ERROR	未接入 flash 芯片
QL_SPI_FLASH_NOT_SUPPORT_ERROR	flash 型号不支持（仅适用于外接 NOR flash）
QL_SPI_FLASH_OP_NOT_SUPPORT_ERROR	flash 型号不支持此操作
QL_SPI_FLASH_ECC_ERROR	NAND flash ECC 校验错误
QL_SPI_FLASH_PROGRAM_ERROR	flash 程序错误
QL_SPI_FLASH_ERASE_ERROR	flash 擦除错误
QL_SPI_FLASH_MUTEX_CREATE_ERROR	flash 互斥锁创建失败错误
QL_SPI_FLASH_MUTEX_LOCK_ERROR	flash 互斥锁上锁错误
QL_SPI_FLASH_SET_GPIO_ERROR	flash 设置引脚错误

3.1.3.1.2. ql_spi_port_e

SPI 总线枚举定义如下：

```
typedef enum
{
    QL_SPI_PORT1,
    QL_SPI_PORT2,
}ql_spi_port_e
```

● 成员

成员	描述
QL_SPI_PORT1	SPI1 总线

QL_SPI_PORT2

SPI2 总线

3.1.3.1.3. ql_spi_clk_e

ECx00U、EGx00U 和 EG91xU 系列模块的通用 SPI 时钟频率枚举定义如下：

```
typedef enum
{
    QL_SPI_CLK_INVALID=-1,
    QL_SPI_CLK_781_25KHZ = 781250,
    QL_SPI_CLK_1_5625MHZ = 1562500,
    QL_SPI_CLK_3_125MHZ = 3125000,
    QL_SPI_CLK_5MHZ = 5000000,
    QL_SPI_CLK_6_25MHZ = 6250000,
    QL_SPI_CLK_10MHZ = 10000000,
    QL_SPI_CLK_12_5MHZ = 12500000,
    QL_SPI_CLK_20MHZ = 20000000,
    QL_SPI_CLK_25MHZ = 25000000,
    QL_SPI_CLK_33_33MHZ = 33000000,
    QL_SPI_CLK_50MHZ_MAX = 50000000,
}ql_spi_clk_e
```

● 成员

成员	描述
QL_SPI_CLK_INVALID	无效参数
QL_SPI_CLK_781_25KHZ	时钟频率为 781.25 kHz
QL_SPI_CLK_1_5625MHZ	时钟频率为 1.5625 MHz
QL_SPI_CLK_3_125MHZ	时钟频率为 3.125 MHz
QL_SPI_CLK_5MHZ	时钟频率为 5 MHz
QL_SPI_CLK_6_25MHZ	时钟频率为 6.25 MHz
QL_SPI_CLK_10MHZ	时钟频率为 10 MHz
QL_SPI_CLK_12_5MHZ	时钟频率为 12.5 MHz
QL_SPI_CLK_20MHZ	时钟频率为 20 MHz
QL_SPI_CLK_25MHZ	时钟频率为 25 MHz
QL_SPI_CLK_33_33MHZ	时钟频率为 33.33 MHz

QL_SPI_CLK_50MHZ_MAX

时钟频率为 50 MHz（最大时钟频率）

ECx00G 和 EGx00G 系列模块通用 SPI 时钟频率枚举定义如下：

```
typedef enum
{
    QL_SPI_CLK_INVALID=-1,
    QL_SPI_CLK_97_656KHZ_MIN = 97656,
    QL_SPI_CLK_100KHZ = 100000,
    QL_SPI_CLK_812_5KHZ = 812500,
    QL_SPI_CLK_1_3MHZ = 1300000,
    QL_SPI_CLK_1_625MHZ = 1625000,
    QL_SPI_CLK_1_5625MHZ = QL_SPI_CLK_1_625MHZ,
    QL_SPI_CLK_2MHZ = 2000000,
    QL_SPI_CLK_3_25MHZ = 3250000,
    QL_SPI_CLK_4_333MHZ = 4333000,
    QL_SPI_CLK_6_6MHZ = 6600000,
    QL_SPI_CLK_11_93MHZ = 11930000,
    QL_SPI_CLK_13MHZ = 13000000,
    QL_SPI_CLK_13_92MHZ = 13920000,
    QL_SPI_CLK_16_7MHZ = 16700000,
    QL_SPI_CLK_20_875MHZ = 20875000,
    QL_SPI_CLK_27_83MHZ = 27830000,
    QL_SPI_CLK_25MHZ = QL_SPI_CLK_27_83MHZ,
    QL_SPI_CLK_41_75MHZ = 41750000,
    QL_SPI_CLK_50MHZ_MAX = 50000000,
}ql_spi_clk_e
```

● 成员

成员	描述
QL_SPI_CLK_INVALID	无效参数
QL_SPI_CLK_97_656KHZ_MIN	时钟频率为 97.656 kHz
QL_SPI_CLK_100KHZ	时钟频率为 100 kHz
QL_SPI_CLK_812_5KHZ	时钟频率为 812.5 kHz
QL_SPI_CLK_1_3MHZ	时钟频率为 1.3 MHz
QL_SPI_CLK_1_625MHZ	时钟频率为 1.625 MHz
QL_SPI_CLK_1_5625MHZ	时钟频率为 1.625 MHz

QL_SPI_CLK_2MHZ	时钟频率为 2 MHz
QL_SPI_CLK_3_25MHZ	时钟频率为 3.25 MHz
QL_SPI_CLK_4_333MHZ	时钟频率为 4.333 MHz
QL_SPI_CLK_6_6MHZ	时钟频率为 6.6 MHz
QL_SPI_CLK_11_93MHZ	时钟频率为 11.93 MHz
QL_SPI_CLK_13MHZ	时钟频率为 13 MHz
QL_SPI_CLK_13_92MHZ	时钟频率为 13.92 MHz
QL_SPI_CLK_16_7MHZ	时钟频率为 16.7 MHz
QL_SPI_CLK_20_875MHZ	时钟频率为 20.875 MHz
QL_SPI_CLK_27_83MHZ	时钟频率为 27.83 MHz
QL_SPI_CLK_25MHZ	时钟频率为 27.83 MHz
QL_SPI_CLK_41_75MHZ	时钟频率为 41.75 MHz
QL_SPI_CLK_50MHZ_MAX	时钟频率为 50 MHz（最大时钟频率）

3.1.3.1.4. ql_spi_input_sel_e

通用 SPI 数据输入引脚枚举定义如下：

```
typedef enum
{
    QL_SPI_DI_0 = 0,
    QL_SPI_DI_1,
    QL_SPI_DI_2,
}ql_spi_input_sel_e
```

● 成员

成员	描述
QL_SPI_DI_0	DI0 为数据输入引脚（暂不支持）
QL_SPI_DI_1	DI1 为数据输入引脚
QL_SPI_DI_2	DI2 为数据输入引脚（暂不支持）

3.1.3.1.5. ql_spi_transfer_mode_e

通用 SPI 传输模式枚举定义如下：

```
typedef enum
{
    QL_SPI_DIRECT_POLLING = 0,
    QL_SPI_DIRECT_IRQ,
    QL_SPI_DMA_POLLING,
    QL_SPI_DMA_IRQ,
}ql_spi_transfer_mode_e
```

● 成员

成员	描述
<code>QL_SPI_DIRECT_POLLING</code>	FIFO 读写，轮询等待。
<code>QL_SPI_DIRECT_IRQ</code>	FIFO 读写，中断通知（暂不支持）
<code>QL_SPI_DMA_POLLING</code>	DMA 读写，轮询等待。
<code>QL_SPI_DMA_IRQ</code>	DMA 读写，中断通知（暂不支持）

备注

ECx00U、EGx00U 和 EG91xU 系列模块使用 DMA 轮询模式时，通用 SPI 与 SD 卡抢占 DMA 通道；若 SD 卡已使用 DMA 通道，通用 SPI 初始化则会失败。ECx00G 和 EGx00G 系列模块无此影响。

3.1.3.1.6. ql_spi_cs_sel_e

通用 SPI 的 CS 引脚枚举定义如下：

```
typedef enum
{
    QL_SPI_CS0 = 0,
    QL_SPI_CS1,
    QL_SPI_CS2,
    QL_SPI_CS3,
}ql_spi_cs_sel_e
```

- 成员

成员	描述
<code>QL_SPI_CS0</code>	CS0 为通用 SPI 的 CS 引脚
<code>QL_SPI_CS1</code>	CS1 为通用 SPI 的 CS 引脚
<code>QL_SPI_CS2</code>	CS2 为通用 SPI 的 CS 引脚（暂不支持）
<code>QL_SPI_CS3</code>	CS3 为通用 SPI 的 CS 引脚（暂不支持）

3.1.3.2. ql_spi_nand_init_ext

该函数用于初始化配置通用 SPI，包括 SPI 总线、时钟频率、传输模式、数据输入引脚和 CS 引脚。调用此函数前，需调用 `ql_pin_set_func()` 设置相关 GPIO 引脚为通用 SPI 功能，详情请参考[文档 \[2\]](#)。

- 函数原型

```
ql_errcode_spi_nand_e ql_spi_nand_init_ext(ql_spi_nand_config_s nand_config)
```

- 参数

`nand_config`:

[In] 通用 SPI 配置参数；详见[第 3.1.3.2.1 章](#)。

- 返回值

详见[第 3.1.3.1.1 章](#)。

备注

若选择了错误的 SPI 总线进行初始化，将导致 NAND flash 读、写、擦除操作失败。因此，需对初始化返回值进行判定，若判定为初始化失败，应避免进行后续读、写、擦除操作。

3.1.3.2.1. ql_spi_nand_config_s

通用 SPI 配置参数结构体定义如下：

```
typedef ql_spi_flash_config_s ql_spi_nand_config_s
```

```
typedef struct
{
```

```

ql_spi_port_e port;
ql_spi_clk_e spiclk;
ql_spi_input_sel_e input_sel;
ql_spi_transfer_mode_e transmode;
ql_spi_cs_sel_e cs;
} ql_spi_flash_config_s

```

● 参数

类型	参数	描述
<i>ql_spi_port_e</i>	<i>port</i>	SPI 总线；详见第3.1.3.1.2章。
<i>ql_spi_clk_e</i>	<i>spiclk</i>	通用 SPI 时钟频率；详见第3.1.3.1.3章。
<i>ql_spi_input_sel_e</i>	<i>input_sel</i>	通用 SP 数据输入引脚；详见第3.1.3.1.4章。
<i>ql_spi_transfer_mode_e</i>	<i>transmode</i>	通用 SPI 传输模式；详见第3.1.3.1.5章。
<i>ql_spi_cs_sel_e</i>	<i>cs</i>	通用 SPI 的 CS 引脚；详见第3.1.3.1.6章。

3.1.3.3. ql_spi_nand_read_devid

该函数用于读取通用 SPI NAND flash 的 JEDEC ID。

● 函数原型

```
ql_errcode_spi_nand_e ql_spi_nand_read_devid(ql_spi_port_e port, unsigned char *mid)
```

● 参数

port:

[In] SPI 总线；详见第3.1.3.1.2章。

mid:

[Out] 通用 SPI NAND flash 的 JEDEC ID。

● 返回值

详见第3.1.3.1.1章。

3.1.3.4. ql_spi_nand_read_devid_ex

该函数用于根据类型读取通用 SPI NAND flash 的 JEDEC ID。

- 函数原型

```
ql_errcode_spi_nand_e ql_spi_nand_read_devid_ex(ql_spi_port_e port, unsigned char
*mid,ql_spi_nand_id_type_e mid_type)
```

- 参数

port:

[In] SPI 总线；详见第3.1.3.1.2 章。

mid:

[Out] 通用 SPI NAND flash 的 JEDEC ID。

mid_type:

[In] NAND flash 的 JEDEC ID 类型；详见第3.1.3.4.1 章。

- 返回值

详见第3.1.3.1.1 章。

3.1.3.4.1. ql_spi_nand_id_type_e

NAND flash 的 JEDEC ID 类型枚举定义如下：

```
typedef enum
{
    QL_NAND_FLASH_ID_16BIT = 0,,
    QL_NAND_FLASH_ID_24BIT,
}ql_spi_nand_id_type_e
```

- 成员

成员	描述
QL_NAND_FLASH_ID_16BIT	16 位 NAND flash ID，即使用 9Fh 命令读取到的 JEDEC ID，由厂商 ID（8 位）和设备 ID（8 位）组成。
QL_NAND_FLASH_ID_24BIT	24 位 NAND flash ID，即使用 9Fh 命令读取到的 JEDEC ID，由厂商 ID（8 位）和设备 ID（16 位）组成。

3.1.3.5. ql_spi_nand_read_page_spare

该函数用于读取通用 SPI NAND flash 的页（2048 字节）和备用区（64 字节）数据。

- 函数原型

```
ql_errcode_spi_nand_e ql_spi_nand_read_page_spare(ql_spi_port_e port, unsigned int
page_addr, unsigned short column_addr, unsigned char *data, int len)
```

- 参数

port:

[In] SPI 总线；详见第3.1.3.1.2章。

page_addr:

[In] 页地址。

column_addr:

[In] 列地址。

data:

[Out] 读取出的缓存数据。

len:

[In] 读取数据长度。单位：字节。

- 返回值

详见第3.1.3.1.1章。

3.1.3.6. ql_spi_nand_write_page_spare

该函数用于写入数据到通用 SPI NAND flash 的页（2048 字节）和备用区（64 字节）。

- 函数原型

```
ql_errcode_spi_nand_e ql_spi_nand_write_page_spare(ql_spi_port_e port, unsigned int
page_addr, unsigned short column_addr, unsigned char *data, int len)
```

- 参数

port:

[In] SPI 总线；详见第3.1.3.1.2章。

page_addr:

[In] 页地址。

column_addr:

[In] 列地址。

data:

[In] 需写入的数据。

len:

[In] 写入数据长度。单位：字节。

- 返回值

详见第3.1.3.1.1章。

3.1.3.7. ql_spi_nand_write_spare

该函数用于写入数据到通用 SPI NAND flash 的备用区（64 字节）。

- 函数原型

```
ql_errcode_spi_nand_e ql_spi_nand_write_spare(ql_spi_port_e port, unsigned int page_addr,
unsigned char *data, int len)
```

- 参数

port:

[In] SPI 总线；详见第3.1.3.1.2章。

page_addr:

[In] 页地址。

data:

[In] 需写入的数据。

len:

[In] 写入数据的长度。单位：字节。

- 返回值

详见第3.1.3.1.1章。

3.1.3.8. ql_spi_nand_read_status

该函数用于读取通用 SPI NAND flash 状态寄存器的值。

● 函数原型

```
ql_errcode_spi_nand_e ql_spi_nand_read_status(ql_spi_port_e port, ql_spi_nand_status_reg_e reg, unsigned char *status)
```

● 参数

port:

[In] SPI 总线；详见第3.1.3.1.2 章。

reg:

[In] 状态寄存器；详见第3.1.3.8.1 章。

status:

[Out] 读取的状态寄存器的值。

● 返回值

详见第3.1.3.1.1 章。

3.1.3.8.1. ql_spi_nand_status_reg_e

状态寄存器枚举定义如下：

```
typedef enum
{
    QL_NAND_FLASH_STATUS_1 = 0,
    QL_NAND_FLASH_STATUS_2,
    QL_NAND_FLASH_STATUS_3,
    QL_NAND_FLASH_STATUS_4,
}ql_spi_nand_status_reg_e
```

● 成员

成员	描述
QL_NAND_FLASH_STATUS_1	状态寄存器 1
QL_NAND_FLASH_STATUS_2	状态寄存器 2
QL_NAND_FLASH_STATUS_3	状态寄存器 3
QL_NAND_FLASH_STATUS_4	状态寄存器 4

3.1.3.9. ql_spi_nand_write_status

该函数用于写入通用 SPI NAND flash 状态寄存器的值。

- 函数原型

```
ql_errcode_spi_nand_e ql_spi_nand_write_status(ql_spi_port_e port, ql_spi_nand_status_reg_e reg, unsigned char status)
```

- 参数

port:

[In] SPI 总线；详见第3.1.3.1.2章。

reg:

[In] 状态寄存器；详见第3.1.3.8.1章。

status:

[In] 写入的状态寄存器的值。

- 返回值

详见第3.1.3.1.1章。

3.1.3.10. ql_spi_nand_erase_block

该函数用于擦除通用 SPI NAND flash 的块。

- 函数原型

```
ql_errcode_spi_nand_e ql_spi_nand_erase_block(ql_spi_port_e port, unsigned int page_addr)
```

- 参数

port:

[In] SPI 总线；详见第3.1.3.1.2章。

page_addr:

[In] 页地址。

- 返回值

详见第3.1.3.1.1章。

3.1.3.11. ql_spi_nand_reset

该函数用于复位通用 SPI NAND flash。

- 函数原型

```
ql_errcode_spi_nand_e ql_spi_nand_reset(ql_spi_port_e port)
```

- 参数

port:

[In] SPI 总线；详见第3.1.3.1.2 章。

- 返回值

详见第3.1.3.1.1 章。

3.1.3.12. ql_spi6_nand_init

该函数用于初始化配置专用 SPI（仅配置 SPI 总线、时钟频率和时钟分频系数）。调用该函数前，需通过 *ql_pin_set_func()* 设置相关 GPIO 引脚为专用 SPI 功能，详情请参考文档 [2]。

- 函数原型

```
ql_errcode_spi_nand_e ql_spi6_nand_init(ql_spi_port_e port, ql_spi6_nand_flash_clk_e clk,
uint8_t clk_div)
```

- 参数

port:

[In] SPI 总线。此参数无实际意义。

clk:

[In] 时钟频率；详见第3.1.3.12.1 章。

clk_div:

[In] 分频系数。范围：1~255。

- 返回值

详见第3.1.3.1.1 章。

备注

需对初始化返回值进行判定，若判定为初始化失败，应避免进行后续读、写、擦除操作。

3.1.3.12.1.ql_spi6_nand_flash_clk_e

ECx00U、EGx00U 和 EG91xU 系列模块的专用 SPI 时钟频率枚举定义如下：

```
typedef enum
{
    QL_SPI6_NAND_FLASH_CLK_SRC_62MHZ = 1,
    QL_SPI6_NAND_FLASH_CLK_SRC_82MHZ,
    QL_SPI6_NAND_FLASH_CLK_SRC_124MHZ,
    QL_SPI6_NAND_FLASH_CLK_SRC_142MHZ,
    QL_SPI6_NAND_FLASH_CLK_SRC_166MHZ,
    QL_SPI6_NAND_FLASH_CLK_SRC_182MHZ,
    QL_SPI6_NAND_FLASH_CLK_SRC_200MHZ,
    QL_SPI6_NAND_FLASH_CLK_SRC_222MHZ,
    QL_SPI6_NAND_FLASH_CLK_SRC_250MHZ,
    QL_SPI6_NAND_FLASH_CLK_SRC_INVALID = 10,
}ql_spi6_nand_flash_clk_e
```

● 成员

成员	描述
QL_SPI6_NAND_FLASH_CLK_SRC_62MHZ	时钟频率为 62 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_82MHZ	时钟频率为 82 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_124MHZ	时钟频率为 124 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_142MHZ	时钟频率为 142 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_166MHZ	时钟频率为 166 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_182MHZ	时钟频率为 182 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_200MHZ	时钟频率为 200 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_222MHZ	时钟频率为 222 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_250MHZ	时钟频率为 250 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_INVALID	无效参数

ECx00G 和 EGx00G 系列模块专用 SPI 时钟频率枚举定义如下：

```
typedef enum
{
    QL_SPI6_NAND_FLASH_CLK_SRC_62MHZ = 0x1,
    QL_SPI6_NAND_FLASH_CLK_SRC_71MHZ = 0x3,
    QL_SPI6_NAND_FLASH_CLK_SRC_83MHZ = 0x5,
    QL_SPI6_NAND_FLASH_CLK_SRC_91MHZ = 0x6,
    QL_SPI6_NAND_FLASH_CLK_SRC_100MHZ = 0x7,
    QL_SPI6_NAND_FLASH_CLK_SRC_111MHZ = 0x8,
    QL_SPI6_NAND_FLASH_CLK_SRC_125MHZ = 0x9,
    QL_SPI6_NAND_FLASH_CLK_SRC_142MHZ = 0xa,
    QL_SPI6_NAND_FLASH_CLK_SRC_166MHZ = 0xb,
    QL_SPI6_NAND_FLASH_CLK_SRC_200MHZ = 0xc,
    QL_SPI6_NAND_FLASH_CLK_SRC_500MHZ = 0xf,
    QL_SPI6_NAND_FLASH_CLK_SRC_INVALID = 0x10,
}ql_spi6_nand_flash_clk_e
```

● 成员

成员	描述
QL_SPI6_NAND_FLASH_CLK_SRC_62MHZ	时钟频率为 62 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_71MHZ	时钟频率为 71 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_83MHZ	时钟频率为 83 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_91MHZ	时钟频率为 91 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_100MHZ	时钟频率为 100 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_111MHZ	时钟频率为 111 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_125MHZ	时钟频率为 125 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_142MHZ	时钟频率为 142 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_166MHZ	时钟频率为 166 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_200MHZ	时钟频率为 200 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_500MHZ	时钟频率为 500 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_INVALID	无效参数

3.1.3.13.ql_spi6_nand_init_ext

该函数用于初始化配置专用 SPI，包括 SPI 总线、时钟频率、时钟分频系数、SPI 模式、延时采样系数和是否使用低速率时钟。调用该函数前，需通过 *ql_pin_set_func()* 设置相关 GPIO 引脚为专用 SPI 功能，详情请参考文档 [2]。

- 函数原型

```
ql_errcode_spi_nand_e ql_spi6_nand_init_ext(ql_spi6_nand_config_s config)
```

- 参数

config:

[In] 专用 SPI 配置参数；详见第 3.1.3.13.1 章。

- 返回值

详见第 3.1.3.1.1 章。

备注

需对初始化返回值进行判定，若判定为初始化失败，应避免进行后续读、写、擦操作。

3.1.3.13.1.ql_spi6_nand_config_s

专用 SPI 配置参数结构体定义如下：

```
typedef ql_spi_flash_config_s ql_spi_nand_config_s
```

```
typedef struct
{
    ql_spi_port_e port;
    ql_spi6_nand_flash_clk_e clk;
    uint8_t clk_div;
    uint8_t quad_mode;
    uint8_t sample_delay;
    uint8_t isslow;
} ql_spi6_nand_config_s
```

● 参数

类型	参数	描述
<i>ql_spi_port_e</i>	<i>port</i>	专用 SPI 总线。此参数无实际意义。
<i>ql_spi6_nand_flash_clk_e</i>	<i>clk</i>	专用 SPI 时钟频率；详见第3.1.3.12.1章。
<i>uint8_t</i>	<i>clk_div</i>	专用 SPI 时钟频率的分频系数。范围：1~255。
<i>uint8_t</i>	<i>quad_mode</i>	专用 SPI 模式。 1 Quad SPI 0 Dual SPI
<i>uint8_t</i>	<i>sample_delay</i>	延时采样系数。范围：0~255。
<i>uint8_t</i>	<i>isslow</i>	是否使用低速率时钟。 1 使用低速率时钟 0 使用快速率时钟 (此参数仅对 ECx00U、EGx00U 和 EG91xU 系列模块有效。)

3.1.3.14.ql_spi6_nand_read_devid

该函数用于读取专用 SPI NAND flash 的 JEDEC ID。

● 函数原型

```
ql_errcode_spi_nand_e ql_spi6_nand_read_devid(ql_spi_port_e port, unsigned char *mid)
```

● 参数

port:

[In] SPI 总线。此参数无实际意义。

mid:

[Out] 专用 SPI NAND flash 的 JEDEC ID。

● 返回值

详见第3.1.3.1.1章。

3.1.3.15.ql_spi6_nand_read_devid_ex

该函数用于根据类型读取专用 SPI NAND flash 的 JEDEC ID。

- 函数原型

```
ql_errcode_spi_nand_e ql_spi6_nand_read_devid_ex(ql_spi_port_e port, unsigned char *mid,
ql_spi_nand_id_type_e mid_type)
```

- 参数

port:

[In] SPI 总线。此参数无实际意义。

mid:

[Out] 专用 SPI NAND flash 的 JEDEC ID。

mid_type:

[In] NAND flash 的 JEDEC ID 类型；详见第3.1.3.4.1 章。

- 返回值

详见第3.1.3.1.1 章。

3.1.3.16.ql_spi6_nand_read_page_spare

该函数用于读取专用 SPI NAND flash 的页（2048 字节）和备用区（64 字节）数据。

- 函数原型

```
ql_errcode_spi_nand_e ql_spi6_nand_read_page_spare(ql_spi_port_e port, unsigned int
page_addr, unsigned short column_addr, unsigned char *data, int len)
```

- 参数

port:

[In] SPI 总线。此参数无实际意义。

page_addr:

[In] 页地址。

column_addr:

[In] 列地址。

data:

[Out] 读取出的缓存数据。

len:

[In] 读取数据长度。单位：字节。

- 返回值

详见第3.1.3.1.1章。

3.1.3.17. ql_spi6_nand_write_page_spare

该函数用于写入数据到专用 SPI NAND flash 的页（2048 字节）和备用区（64 字节）。

- 函数原型

```
ql_errcode_spi_nand_e ql_spi6_nand_write_page_spare(ql_spi_port_e port, unsigned int
page_addr, unsigned short column_addr, unsigned char *data, int len)
```

- 参数

port:

[In] SPI 总线。此参数无实际意义。

page_addr:

[In] 页地址。

column_addr:

[In] 列地址。

data:

[In] 需写入的数据。

len:

[In] 写入数据长度。单位：字节。

- 返回值

详见第3.1.3.1.1章。

3.1.3.18. ql_spi6_nand_write_spare

该函数用于写入数据到专用 SPI NAND flash 的备用区（64 字节）。

- 函数原型

```
ql_errcode_spi_nand_e ql_spi6_nand_write_spare(ql_spi_port_e port, unsigned int page_addr,
unsigned char *data, int len)
```

- 参数

port:

[In] SPI 总线。此参数无实际意义。

page_addr:

[In] 页地址。

data:

[In] 需写入的数据。

len:

[In] 写入数据长度。单位：字节。

- 返回值

详见第3.1.3.1.1章。

3.1.3.19.ql_spi6_nand_read_status

该函数用于读取专用 SPI NAND flash 状态寄存器的值。

- 函数原型

```
ql_errcode_spi_nand_e ql_spi6_nand_read_status(ql_spi_port_e port, ql_spi_nand_status_reg_e reg, unsigned char *status)
```

- 参数

port:

[In] SPI 总线。此参数无实际意义。

reg:

[In] 状态寄存器；详见第3.1.3.8.1章。

status:

[Out] 读取的状态寄存器的值。

- 返回值

详见第3.1.3.1.1章。

3.1.3.20.ql_spi6_nand_write_status

该函数用于写入专用 SPI NAND flash 状态寄存器的值。

- 函数原型

```
ql_errcode_spi_nand_e ql_spi6_nand_write_status(ql_spi_port_e port, ql_spi_nand_status_reg_e reg, unsigned char status)
```

- 参数

port:

[In] SPI 总线。此参数无实际意义。

reg:

[In] 状态寄存器；详见第 3.1.3.8.1 章。

status:

[In] 写入的状态寄存器的值。

- 返回值

详见第 3.1.3.1.1 章。

3.1.3.21. ql_spi6_nand_erase_block

该函数用于擦除专用 SPI NAND flash 的块。

- 函数原型

```
ql_errcode_spi_nand_e ql_spi6_nand_erase_block(ql_spi_port_e port, unsigned int page_addr)
```

- 参数

port:

[In] SPI 总线。此参数无实际意义。

page_addr:

[In] 页地址。

- 返回值

详见第 3.1.3.1.1 章。

3.1.3.22. ql_spi6_nand_reset

该函数用于复位专用 SPI NAND flash。

- 函数原型

```
ql_errcode_spi_nand_e ql_spi6_nand_reset(ql_spi_port_e port)
```

- 参数

port:

[In] SPI 总线。此参数无实际意义。

- 返回值

详见第3.1.3.1.1章。

3.2. SPI NAND Flash 文件系统挂载 API

3.2.1. 头文件

SPI NAND flash 文件系统挂载 API 头文件为 *ql_api_fs_nand_flash.h*，位于 SDK 包的 *components\ql-kernel\incl* 目录下。若无特别说明，本文档所述头文件均在该目录下。

3.2.2. 函数概览

表 3：函数概览

函数	说明
<i>ql_nand_flash_mkfs()</i>	将 SPI NAND flash 格式化为 FATFS 文件系统格式
<i>ql_nand_flash_umount()</i>	从 FATFS 文件系统中卸载 SPI NAND flash 设备
<i>ql_nand_flash_mount()</i>	挂载 SPI NAND flash 设备到 FATFS 文件系统
<i>ql_nand_get_hw_info()</i>	获取 SPI NAND flash 的硬件信息

3.2.3. 函数详解

3.2.3.1. ql_nand_flash_mkfs

该函数用于将 SPI NAND flash 格式化为 FATFS 文件系统格式。格式化成功后，用文件系统读取 NAND flash 时，将显示为空。

● 函数原型

```
ql_errcode_nand_flash_e ql_nand_flash_mkfs(uint8_t opt)
```

● 参数

opt:

[In] 格式化后的目标格式。

0x01 FM_FAT (FAT12 或 FAT16)

0x02 FM_FAT32

● 返回值

详见第3.2.3.1.1章。

备注

需根据 SPI NAND flash 的块数量选择格式化类型。文件系统规定的块数量与格式化类型的对应关系如下:

- 若块数量在 1024~8220 之间, 仅支持格式化类型为 FM_FAT (FAT12);
- 若块数量在 8229~4190430 之间, 可选择格式化类型为 FM_FAT (FAT16);
- 若块数量在 66130~67048580 之间, 可选择格式化类型为 FM_FAT32。

因此,

- 若块数量小于 66130, 只能将其格式化为 FM_FAT (FAT12 或 FAT16) 格式;
- 若块数量在 66130~4190430 之间, 可以将其格式化为 FM_FAT32 或 FM_FAT (FAT16) 格式;
- 若块数量大于或等于 4190431, 只能将其格式化为 FM_FAT32 格式。

例如, 1 GB 大小的 NAND flash 块数量为 60928, 其格式化类型为 FM_FAT (FAT12 或 FAT16), 而 2 GB 大小的 NAND flash 块数量大于 66130, 可以格式化为 FM_FAT32 或 FM_FAT (FAT16) 格式。

3.2.3.1.1. ql_errcode_nand_flash_e

SPI NAND flash 文件系统挂载结果码枚举定义如下:

```
typedef enum
{
    QL_NAND_FLASH_SUCCESS                = QL_SUCCESS,
    QL_NAND_FLASH_INVALID_PARAM_ERR      =
1|QL_NAND_FLASH_ERRCODE_BASE,
    QL_NAND_FLASH_MOUNT_ERR,
    QL_NAND_FLASH_MKFS_ERR,
    QL_NAND_FLASH_NOT_MOUNT,
    QL_NAND_FLASH_MUTEX_CREATE_ERR,
    QL_NAND_FLASH_MUTEX_LOCK_ERR,
} ql_errcode_nand_flash_e
```

- 成员

成员	描述
<code>QL_NAND_FLASH_SUCCESS</code>	函数执行成功
<code>QL_NAND_FLASH_INVALID_PARAM_ERR</code>	输入参数错误
<code>QL_NAND_FLASH_MOUNT_ERR</code>	flash 挂载错误
<code>QL_NAND_FLASH_MKFS_ERR</code>	flash 格式化错误
<code>QL_NAND_FLASH_NOT_MOUNT</code>	flash 未挂载错误
<code>QL_NAND_FLASH_MUTEX_CREATE_ERR</code>	flash 互斥锁创建失败错误
<code>QL_NAND_FLASH_MUTEX_LOCK_ERR</code>	flash 互斥锁上锁错误

3.2.3.2. ql_nand_flash_umount

该函数用于从 FATFS 文件系统中卸载 SPI NAND flash 设备。

- 函数原型

```
ql_errcode_nand_flash_e ql_nand_flash_umount(void)
```

- 参数

无

- 返回值

详见第3.2.3.1.1章。

3.2.3.3. ql_nand_flash_mount

该函数用于挂载 SPI NAND flash 设备到 FATFS 文件系统。

- 函数原型

```
ql_errcode_nand_flash_e ql_nand_flash_mount(void)
```

- 参数

无

- 返回值

详见第3.2.3.1.1章。

3.2.3.4. ql_nand_get_hw_info

该函数用于获取 SPI NAND flash 的硬件信息。

- 函数原型

```
ql_errcode_nand_flash_e ql_nand_get_hw_info(ql_nand_hw_info_t *info)
```

- 参数

info:

[Out] SPI NAND flash 的硬件信息；详见第3.2.3.4.1章。

- 返回值

详见第3.2.3.1.1章。

3.2.3.4.1. ql_nand_hw_info_t

SPI NAND flash 的硬件信息结构体定义如下：

```
typedef struct
{
    uint32_t mid;
    uint32_t blknum;
    uint32_t blksize;
} ql_nand_hw_info_t
```

- 参数

类型	参数	描述
uint32_t	<i>mid</i>	厂商 ID
uint32_t	<i>blknum</i>	设备的块数量
uint32_t	<i>blksize</i>	设备的一个块的大小

4 示例

本章节主要介绍如何在应用层使用上述 API 进行通用（4 线）SPI NAND flash 和 SPI NAND flash 文件系统挂载的开发以及简单调试。

备注

将 TE-A 上的 NOR flash 更换成 NAND flash 进行测试时，如使用 Standard SPI 或 Dual SPI 方式驱动 NAND flash，需注意：

1. 建议断开 SIO2 和 SIO3 引脚；
2. 如保持 NAND flash 的 SIO2、SIO3 引脚连接，需要将 SIO2 和 SIO3 配置为上拉推挽输出。

4.1. 开发示例

4.1.1. 通用 SPI NAND Flash API 开发示例

模块 QuecOpen CSDK 代码中提供了通用 SPI NAND flash API 的示例文件 `components\ql-application\spi_nand_flash\spi_nand_flash_demo.c`。该示例中主要包含通用 SPI NAND flash 初始化、状态寄存器值读和写、页和备份区读和写、擦除块、JEDEC ID 读取及复位等操作。示例入口函数为 `ql_spi_nand_flash_demo_init()`，如下图所示。

```
qlosStatus ql_spi_nand_flash_demo_init(void)
{
    qlosStatus err = QL_OSI_SUCCESS;
    err = ql_rtos_task_create(&spi_nand_flash_demo_task, SPI_NAND_FLASH_DEMO_TASK_STACK_SIZE, SPI_NAND_FLASH_DEMO_TASK_Prio, "ql_spi_nand", ql_spi_nand_flash_demo_task_pthread, NULL, SPI_NAND_FLASH_DEMO_TASK_EVENT_CNT);
    if(err != QL_OSI_SUCCESS)
    {
        QL_SPI_NAND_LOG("demo_task created failed");
        return err;
    }
    return err;
}
```

图 4：入口函数 `ql_spi_nand_flash_demo_init()`

如下图所示，上述示例已在 `ql_init_demo_thread` 线程中默认不启动。如需设置示例自启动，取消 `ql_spi_nand_flash_demo_init()` 的注释即可。

```
#ifndef QL_APP_FEATURE_SPI_NAND_FLASH
//ql_spi_nand_flash_demo_init();
#endif
```

图 5：通用 SPI NAND Flash API 示例默认不启动

4.1.2. SPI NAND Flash 文件系统挂载 API 开发示例

模块的 QuecOpen CSDK 中提供了 SPI NAND flash 文件系统挂载 API 示例文件 `lcomponents\ql-application\fs_nand_flash\fs_nand_flash_demo.c`。该示例中主要包含 SPI NAND flash 初始化、挂载 FATFS 文件系统、SPI NAND flash 格式化、通过文件系统进行读写及卸载 FATFS 文件系统等操作。示例入口函数为 `ql_fs_nand_flash_demo_init()`，如下图所示。

```
QIosStatus ql_fs_nand_flash_demo_init(void)
{
    QIosStatus err = QL_OSI_SUCCESS;
    err = ql_rtos_task_create(&fs_nand_flash_demo_task, FS_NAND_FLASH_DEMO_TASK_STACK_SIZE, FS_NAND_FLASH_DEMO_TASK_Prio, "ql_nand_flash", ql_fs_nand_flash_demo_task_thread, NULL, FS_NAND_FLASH_DEMO_TASK_EVENT_CNT);
    if(err != QL_OSI_SUCCESS)
    {
        QL_NAND_FLASH_LOG("demo_task_created_failed");
        return err;
    }
    return err;
}
```

图 6：入口函数 `ql_fs_nand_flash_demo_init()`

如下图所示，上述示例已在 `ql_init_demo_thread` 线程中默认不启动。如需设置示例自启动，取消 `ql_fs_nand_flash_demo_init()` 的注释即可。

```
#ifndef QL_APP_FEATURE_FS_NAND_FLASH
    //ql_fs_nand_flash_demo_init();
#endif
```

图 7：SPI NAND Flash 文件系统挂载 API 示例默认不启动

4.2. 功能调试

4.2.1. 通用 SPI NAND Flash API 功能调试

调试 SPI NAND flash 功能前，需在 LTE OPEN EVB 上外接 NAND flash 芯片。

编译固件版本烧录到模块中，使用 USB 线连接 LTE OPEN EVB 板的 USB 端口和 PC，USB AP Log Port 和 USB DIAG Port 主要用于显示系统调试信息。通过 cooltools 或 ArmLogel 工具抓取 log 后，可以查看上述应用示例的相关信息，log 的抓取方法请参考文档 [3]和[4]。

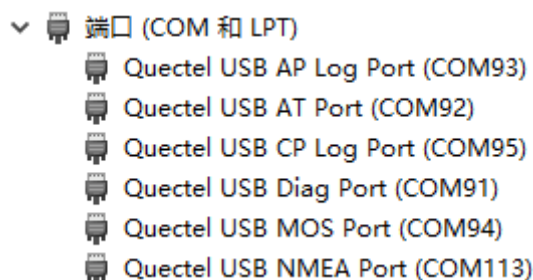


图 8：COM 设备（ECx00U 系列/EGx00U 系列/EG91xU 系列模块）



图 9: COM 设备 (ECx00G 系列/EGx00G 系列模块)

备注

ECx00U 系列、EGx00U 系列和 EG91xU 系列模块使用 cooltools 工具抓取 log，端口使用 USB AP Log Port，抓取方法请参考文档 [3]。ECx00G 系列和 EGx00G 系列模块使用 ArmLogel 工具抓取 log，端口使用 USB DIAG Port，抓取方法请参考文档 [4]。

模块开机后自动启动 `ql_spi_nand_flash_demo_init()`。通过比较通用 SPI NAND flash 页和备用区读写后的数据，判断读写是否操作成功；擦除操作后进行读取操作，若读取出的数据全为 FF，则可判断擦除操作成功。

如函数执行失败，可根据相应 log 信息查看失败原因。

Index	Received	Tick	Level	Description
71	09:58:44.532	41192	QOPN/I	[ql_spi][ql_spi_write, 494] write success
72	09:58:44.532	41194	QOPN/I	[ql_spi_nand][ql_spi_nand_wait_write_finish, 244] status=2
73	09:58:44.532	41194	QOPN/I	[ql_spi][ql_spi_write, 494] write success
74	09:58:44.532	41194	QOPN/I	[ql_spi_nand][ql_spi_nand_wait_write_finish, 244] status=0
99	09:58:44.563	41391	QOPN/I	[ql_spi][ql_spi_write, 494] write success
100	09:58:44.563	41391	QOPN/I	[ql_spi][ql_spi_write, 494] write success
103	09:58:44.579	41683	QOPN/I	[ql_spi_nand][ql_spi_nand_read_dev_id, 534] mid =efbf22
104	09:58:44.579	41684	QOPN/I	[ql_spi_nand_demo][ql_spi_nand_flash_demo_task_pthread, 186] sr1 = 0
105	09:58:44.579	41684	QOPN/I	[ql_spi_nand_demo][ql_spi_nand_flash_demo_task_pthread, 195] sr2 = 18
106	09:58:44.579	41684	QOPN/I	[ql_spi_nand_demo][ql_spi_nand_flash_demo_task_pthread, 204] sr3 = 0
107	09:58:44.579	41685	QOPN/I	[ql_spi_nand_demo][ql_spi_nand_flash_demo_task_pthread, 213] sr4 = 0
108	09:58:44.579	41685	QOPN/I	[ql_spi_nand_demo][ql_spi_nand_flash_demo_task_pthread, 222] a2 = 0xaaaaaaaa

图 10: Log 信息 (通用 SPI NAND Flash)

4.2.2. SPI NAND Flash 文件系统挂载 API 功能调试

调试 SPI NAND flash 文件系统挂载功能前，需在 LTE OPEN EVB 上外接 NAND flash 芯片。

编译固件版本烧录到模块中，使用 USB 线连接 LTE OPEN EVB 板的 USB 端口和 PC，USB AP Log Port 和 USB DIAG Port 主要用于显示系统调试信息。通过 cooltools 或 ArmLogel 工具抓取 log 后，可以查看上述应用示例的相关信息，log 的抓取方法请参考文档 [3]和[4]。不同模块的 COM 设备图，见图8和图9。

备注

通过示例文件 `fs_nand_flash_demo.c` 中的 `QL_SPI_TPYE` 宏选择一种 SPI 模式 (即 Standard SPI、Dual SPI 或 Quad SPI，如下图所示) 后，示例文件将根据选择的 SPI 模式调整初始化流程。


```
#define QL_STANDARD_SPI 0 //Standard SPI: SCLK, CS#, SI, SO
#define QL_QUAD_SPI 1 //Quad SPI: SCLK, CS#, SI00, SI01, SI02, SI03
#define QL_DUAL_SPI 2 //Dual SPI: SCLK, CS#, SI00, SI01

#define QL_SPI_TPYE QL_QUAD_SPI
```

模块开机后自动启动 `ql_fs_nand_flash_demo_init()`。通过 log 信息可以看到 SPI NAND flash 文件系统挂载的相关信息。log 端口调试信息如下图所示。

```
[14:53:33.240] [19403] [ 1020] [ ] [ ] [ ] QOPN/I : ql_spi_nand_read_devid_ex 592 mid[0] =efbf
[14:53:33.240] [19404] [ 1021] [ ] [ ] [ ] QOPN/I : ql_spi_nand_read_devid_ex 592 mid[1] =efbf22
[14:53:33.240] [19413] [ 1022] [ ] [ ] [ ] QUEC/I : ql_ftl_init_cache_block:599 position=0, block=0
[14:53:33.240] [19549] [ 1023] [ ] [ ] [ ] QUEC/I : ql_ftl_init_cache_block:665 position=0, used_bit=64
[14:53:33.240] [19553] [ 1024] [ ] [ ] [ ] QUEC/I : ql_ftl_init_cache_block:599 position=1, block=1
[14:53:33.240] [19685] [ 1025] [ ] [ ] [ ] QUEC/I : ql_ftl_init_cache_block:665 position=1, used_bit=62
[14:53:33.240] [19689] [ 1026] [ ] [ ] [ ] QUEC/I : ql_ftl_init_cache_block:599 position=2, block=2
[14:53:33.240] [19820] [ 1027] [ ] [ ] [ ] QUEC/I : ql_ftl_init_cache_block:665 position=2, used_bit=64
[14:53:33.240] [19824] [ 1028] [ ] [ ] [ ] QUEC/I : ql_ftl_init_cache_block:599 position=3, block=3
[14:53:33.240] [19952] [ 1029] [ ] [ ] [ ] QUEC/I : ql_ftl_init_cache_block:665 position=3, used_bit=64
[14:53:33.240] [19957] [ 1030] [ ] [ ] [ ] QUEC/I : ql_ftl_init_cache_block:599 position=4, block=4
[14:53:33.299] [20085] [ 1031] [ ] [ ] [ ] QUEC/I : ql_ftl_init_cache_block:665 position=4, used_bit=64
[14:53:33.524] [24717] [ 1032] [ ] [ ] [ ] QUEC/I : ql_ftl_table_create:862 nand read phy total=2048, bad_blocknum=0, good_blocknum=2048
[14:53:33.588] [24717] [ 1033] [ ] [ ] [ ] QUEC/I : ql_ftl_table_create:863 data_block_num=1904, free_blocknum=139, cache_blocknum=5
[14:53:33.741] [27484] [ 1034] [ ] [ ] [ ] QUEC/I : ql_ftl_exchange_data_free_node:1885 change old-b:-65,new-b:2046
[14:53:33.741] [27484] [ 1035] [ ] [ ] [ ] QUEC/I : ql_ftl_mark_cache_block_position:704 mark block_position=0x1
[14:53:33.741] [27493] [ 1036] [ ] [ ] [ ] QUEC/I : ql_ftl_exchange_cache_data:1345 exchange cache position=1, count=1
[14:53:33.741] [27609] [ 1037] [ ] [ ] [ ] QUEC/I : ql_ftl_exchange_cache_data:1368 cache old-b:-1,new-b:2047
[14:53:33.741] [27611] [ 1038] [ ] [ ] [ ] QUEC/I : ql_mount_nand_flash:288 ql_mount_nand_flash: block count: 121856, block size: 2048
[14:53:33.741] [27676] [ 1058] [ ] [ ] [ ] QUEC/I : ql_fs_mount_nand_flash:328 mount_nand_flash ret: 1
[14:53:33.741] [27677] [ 1059] [ ] [ ] [ ] QOPN/I : ql_fs_nand_flash_demo_task_thread 170 mid=efbf22, total_size = 249561088
[14:53:33.741] [27683] [ 1060] [ ] [ ] [ ] QUEC/I : ql_nand_block_read:146 read (0x80c1deb4) nr/4096 count/1
[14:53:33.741] [27704] [ 1061] [ ] [ ] [ ] QOPN/I : ql_mkdir 673 Dir is already exist
[14:53:33.741] [27704] [ 1062] [ ] [ ] [ ] QOPN/I : ql_fs_nand_flash_demo_task_thread 182 dir exist, not create
[14:53:33.741] [27707] [ 1063] [ ] [ ] [ ] QOPN/I : ql_get_file_info 690 get node count failed, or no file in list
[14:53:33.741] [27712] [ 1064] [ ] [ ] [ ] QUEC/I : ql_nand_block_read:146 read (0x80c1deb4) nr/4097 count/1
[14:53:33.741] [27735] [ 1065] [ ] [ ] [ ] QUEC/I : ql_nand_block_write:175 write (0x80c1deb4) nr/4097 count/1
[14:53:33.741] [28067] [ 1066] [ ] [ ] [ ] QUEC/I : ql_ftl_exchange_data_free_node:1885 change old-b:-2041,new-b:2045
[14:53:33.741] [28068] [ 1067] [ ] [ ] [ ] QUEC/I : ql_nand_block_read:146 read (0x80c1deb4) nr/3620 count/1
[14:53:33.741] [28088] [ 1068] [ ] [ ] [ ] QUEC/I : ql_nand_block_erase:101 nand erase block start num = 4098, count = 1
[14:53:33.750] [28089] [ 1069] [ ] [ ] [ ] QUEC/I : ql_ftl_erase_continue_page:2335 head_num: 1, tail_num:0, mid_num:0, erase_block_num:0
[14:53:33.750] [28305] [ 1070] [ ] [ ] [ ] QUEC/I : ql_ftl_exchange_data_free_node:1885 change old-b:-2045,new-b:2044
[14:53:33.814] [28306] [ 1071] [ ] [ ] [ ] QUEC/I : ql_nand_block_write:175 write (0x80c1deb4) nr/3620 count/1
[14:53:33.838] [29596] [ 1072] [ ] [ ] [ ] QUEC/I : ql_ftl_exchange_data_free_node:1885 change old-b:-2042,new-b:2043
[14:53:33.838] [29596] [ 1073] [ ] [ ] [ ] QUEC/I : ql_nand_block_write:175 write (0x80c1deb4) nr/3858 count/1
[14:53:33.838] [29619] [ 1074] [ ] [ ] [ ] QUEC/I : ql_nand_block_read:146 read (0x80c1deb4) nr/4097 count/1
[14:53:33.838] [29640] [ 1075] [ ] [ ] [ ] QOPN/I : ql_fopen 169 open file EXXNAND:/test_dir/test_file1.txt, fd 4099
[14:53:33.838] [29649] [ 1076] [ ] [ ] [ ] QUEC/I : ql_nand_block_read:146 read (0x80c1deb4) nr/3620 count/1
[14:53:33.838] [29672] [ 1078] [ ] [ ] [ ] QOPN/I : ql_fs_nand_flash_demo_task_thread 222 file read result is 0123456789ABCDEFHGHIKLMNOPQRSTUVWXYZ
[14:53:33.838] [29674] [ 1079] [ ] [ ] [ ] QUEC/I : ql_nand_block_write:175 write (0x80c1e97c) nr/4098 count/1
[14:53:33.892] [29698] [ 1080] [ ] [ ] [ ] QUEC/I : ql_nand_block_write:175 write (0x80c1deb4) nr/3620 count/1
[14:53:33.900] [30893] [ 1081] [ ] [ ] [ ] QUEC/I : ql_ftl_exchange_data_free_node:1885 change old-b:-2043,new-b:2039
[14:53:33.900] [30893] [ 1082] [ ] [ ] [ ] QUEC/I : ql_nand_block_write:175 write (0x80c1deb4) nr/3858 count/1
[14:53:33.900] [30921] [ 1083] [ ] [ ] [ ] QUEC/I : ql_nand_block_read:146 read (0x80c1deb4) nr/4097 count/1
[14:53:33.923] [30941] [ 1084] [ ] [ ] [ ] QUEC/I : ql_nand_block_write:175 write (0x80c1deb4) nr/4097 count/1
[14:53:33.923] [31179] [ 1085] [ ] [ ] [ ] QUEC/I : ql_ftl_exchange_data_free_node:1885 change old-b:-2044,new-b:2038
[14:53:33.967] [31181] [ 1086] [ ] [ ] [ ] QUEC/I : ql_nand_block_write:175 write (0x80c1deb4) nr/64 count/1
[14:53:33.967] [31515] [ 1087] [ ] [ ] [ ] QUEC/I : ql_ftl_exchange_data_free_node:1885 change old-b:-2040,new-b:2037
[14:53:33.967] [31516] [ 1088] [ ] [ ] [ ] ql_fclose 264 close file fd 4099
[14:53:33.967] [31516] [ 1089] [ ] [ ] [ ] QOPN/I : ql_fs_nand_flash_demo_task_thread 242 ql_rtos_task_delete
[14:53:34.013] [31518] [ 1090] [ ] [ ] [ ] QOPN/I : ql_rtos_task_delete 191 remove task 80C03CA8 ok
```

图 11: Log 信息（SPI NAND Flash 文件系统挂载）

5 附录 参考文档及术语缩写

表 4: 参考文档

文档名称
[1] Quectel_LTE_Standard(U)系列_QuecOpen(SDK)_CSDK_快速开发指导
[2] Quectel_LTE_Standard(U)系列_QuecOpen(SDK)_GPIO_API_参考手册
[3] Quectel_ECx00U&EGx00U&EG91xU 系列_QuecOpen(SDK)_Log_抓取指导
[4] Quectel_ECx00G&EGx00G_QuecOpen(SDK)_Log 抓取指导

表 5: 术语缩写

缩写	英文全称	中文全称
API	Application Programming Interface	应用程序接口
CS	Chip Select	片选
DI	Data Input	数据输入
DMA	Direct Memory Access	直接存储器访问
ECC	Error Correcting Code	错误检查和纠正
EVB	Evaluation Board	评估板
FIFO	First In First Out	先进先出
GPIO	General-Purpose Input/Output	通用型输入/输出
ID	Identifier	标识符
IoT	Internet of Things	物联网
JEDEC	Joint Electron Device Engineering Council	电子器件工程联合委员会
LTE	Long-Term Evolution	长期演进

NAND	NON-AND	与非
PC	Personal Computer	个人计算机
RTOS	Real-Time Operating System	实时操作系统
SD	Secure Digital Card	安全数字卡
SDK	Software Development Kit	软件开发工具包
SPI	Serial Peripheral Interface	串行外设接口
USB	Universal Serial Bus	通用串行总线